

Programming with proofs and explicit contexts

[Extended Abstract]

Brigitte Pientka
School of Computer Science
McGill University
Montreal, Canada
bpientka@cs.mcgill.ca

Joshua Dunfield
School of Computer Science
McGill University
Montreal, Canada
joshua@cs.mcgill.ca

ABSTRACT

This paper explores a new point in the design space of functional programming: functional programming with dependently-typed higher-order data structures described in the logical framework LF. This allows us to program with proofs as higher-order data. We present a decidable bidirectional type system that distinguishes between dependently-typed data and computations. To support reasoning about open data, our foundation makes contexts explicit. This provides us with a concise characterization of open data, which is crucial to elegantly describe proofs. In addition, we present an operational semantics for this language based on higher-order pattern matching for dependently typed objects. Based on this development, we prove progress and preservation.

Categories and Subject Descriptors

D.3.1 [Programming Languages]: Formal Definitions and Theory

General Terms

Theory, Languages

Keywords

Type theory, Dependent types, Logical frameworks

1. INTRODUCTION

Various forms of dependent types have found their way into mainstream functional programming languages to allow programmers to express stronger properties about their programs [30, 26, 2, 29, 16]. In this paper, we explore a new point in the design space of functional programming with dependent types where we can analyze and manipulate dependently-typed higher-order data described in the logical framework LF [9]. LF provides a rich meta-language for describing formal systems defined by axioms and inference

rules (such as a type system, a logic, etc.) together with proofs within these systems (such as a typing derivation, a proof of a proposition, etc.). Its strength and elegance comes from supporting encodings based on higher-order abstract syntax (HOAS), in which binders in the object language are represented as binders in the meta-language. For example, the formula $\forall x.(x = 1) \supset \neg(x = 0)$ is represented as `forall  $\lambda x$ . (eq x (Suc Zero)) imp (not (eq x Zero))`. This simple, but powerful idea avoids the need to implement common and tricky machinery for variables, such as capture-avoiding substitution and renaming.

However, implementing and verifying proofs about HOAS encodings has been notoriously difficult. There are two distinct challenges. First, encodings based on HOAS require us to recursively traverse binders and describe open data objects within a context. Second, dependent types add another degree of complexity: data objects themselves can refer to types and the structure of types can be observed.

Programming with HOAS has recently received widespread attention, although most work has focused on the simply typed setting [5, 21, 25, 10]. Only a few approaches have been considered in the dependently-typed setting. Despeyroux and Leleu [4, 3] extended previous work by Despeyroux et al. [5] which provided a type-theoretic foundation for primitive recursive programming with HOAS. The Delphin language [23] extends these ideas to provide general recursion over HOAS encodings as well as dependent types.

In this paper, we present an alternative to these approaches, extending the first author's previous work on programming with HOAS encodings in the simply typed setting [21] to dependent types. As in that work, we use contextual modal types [15] to separate HOAS data from computations about them. Open data M is characterized by the contextual modal type $A[\Psi]$ where M has type A in the context Ψ . The object M is closed with respect to the context Ψ , so M can refer to variables declared in Ψ . For example, the object `eq x (Suc Zero)` has type `o[x:nat]` (where `o` describes propositions). Since we want to allow recursion over open data and the local context Ψ associated with the type A may grow, our foundation supports *context variables*, which allow us to abstract over contexts. Just as types classify terms, and kinds classify types, we use *context schemas* to classify and characterize contexts.

In this paper, we revisit the key concepts from the simply-typed setting [21] and generalize them to dependent types. While extending the data language with dependent types follows from previous work [15], the generalization of context schemas to dependent types and, especially, the integra-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
Copyright 200X ACM X-XXXXX-XX-X/XX/XX ...\$5.00.

tion of dependently typed higher-order data into the computation language are novel. Our discussion of dependent types will highlight the fact that the simply-typed foundation scales nicely to this richer setting. This lays the foundation for programming with proofs and explicit contexts. Inductive proofs about formal systems can be implemented as recursive functions, and case analysis in the inductive proof corresponds to analyzing dependently-typed higher-order data via pattern matching. We call this language Beluga.

We make the following contributions:

First, we present a syntax-directed decidable type system for dependently-typed open data. Our presentation only admits data objects in canonical form because only those represent meaningful data. This is achieved following ideas due to Watkins et al.[28] and Nanevski et al.[15]. Thus, exotic terms that do not represent legal data objects are prevented. Our framework supports explicit context variables to abstract over the concrete context and parameterizes computations by contexts, classified by context schemas. While context schemas have been present in the simply-typed setting, characterizing contexts in the presence of dependent types is more complex.

Second, we present a type system for computation that allows recursion and pattern matching on dependently-typed open data. This type system is also syntax-directed and decidable. It allows dependent types, but only types indexed by data objects, not arbitrary, possibly recursive computations. Because data objects are in canonical form, equality between two data objects is easily decided by syntactic equality. This is crucial to achieve a decidable type system. The language we describe is intended as an internal language, into which a source level language will be compiled. It clearly distinguishes between *implicit* data objects that occur in a dependent type and may be kept implicit in the actual source program, and *explicit* data objects that are recursively analyzed with higher-order pattern matching [14, 17] and are explicit in the source program. Intuitively, implicit data arguments are the index objects that are reconstructed when translating a source language to this internal language. In addition, our type system is unique in the sense that constraints due to dependent types are resolved eagerly during type checking using higher-order pattern unification, leading to an elegant decidable algorithm.

Finally, we present a small-step operational semantics based on higher-order pattern matching for dependently-typed objects [17, 22] and we prove progress and preservation for our language with respect to this semantics.

This paper is a first important step in laying the type-theoretic foundation for programming with proofs and explicit contexts. Our foundation ensures that contexts are well-formed according to a user-specified schema, and we have formulated a coverage checking algorithm [6] for dependently-typed open data. At this point, the only missing piece is to verify that a given function terminates—we leave this for a separate paper. As such, our work may be thought of as an alternative to Twelf [19], an implementation of the logical framework LF [9] in which proofs about formal systems are encoded via relations, and to Delphin [23] (which, like our system, implements proofs via functions).

2. MOTIVATION

To motivate the problem, we consider a program that

counts the free occurrences of some variable x in a formula. For example, the formula $\forall y.(x = y) \supset (\text{succ } y = \text{succ } x)$ has two free occurrences of x . The data language here is first-order logic with quantification over natural numbers. We begin by defining a type o for propositions and we use higher-order abstract syntax to model the binder in the universal quantifier.

```

nat : type.
Zero : nat.
Suc : nat → nat.

o : type.
eq : nat → nat → o.
imp : o → o → o.
forall : (nat → o) → o.

```

2.1 Counting free variable occurrences

We will approach this problem top-down and first consider the function `cntV` which will recursively analyze formulas using pattern matching. When it reaches the proposition `eq`, it will call a second function `cntVN`. The modal type $\text{o}[x:\text{nat}, y:\text{nat}]$ describes a formula that can refer to the variables x and y . An element of this type is for example the formula $((\text{eq } x \ y) \text{ imp } (\text{eq } (\text{Suc } x) (\text{Suc } y)))$.

When analyzing a formula with a universal quantifier, the set of free variables grows. Hence, we need to abstract over the contexts in which the formula makes sense. Context variables ψ provide this functionality. The function `cntV` takes a context ψ of natural numbers, a formula f , and returns an integer. Just as types classify data objects and kinds classify types, we introduce *schemas* to classify contexts. In the type declaration for the function `cntV` we say that the context variable ψ has the schema $(\text{nat})^*$, meaning that ψ stands for a data-level context $x_1:\text{nat}, \dots, x_n:\text{nat}$.

```

ctx schema natCtx = (nat)*
rec cntV :  $\Pi \psi:\text{natCtx}.\text{o}[\psi, x:\text{nat}] \rightarrow \text{int} =
\Lambda \psi \Rightarrow \text{fin } f \Rightarrow \text{case } f \text{ of}
\text{box } (\psi, x. \text{imp } U[\text{id}_\psi, x]) \ W[\text{id}_\psi, x] \Rightarrow
\text{cntV } [\psi] \ \text{box } (\psi, x. U[\text{id}_\psi, x])
+ \text{cntV } [\psi] \ \text{box } (\psi, x. W[\text{id}_\psi, x])
| \text{box } (\psi, x. \text{forall } (\lambda y. U[\text{id}_\psi, x, y])) \Rightarrow
\text{cntV } [\psi, y:\text{nat}] \ \text{box } (\psi, y, x. U[\text{id}_\psi, x, y])
| \text{box } (\psi, x. \text{eq } U[\text{id}_\psi, x] \ V[\text{id}_\psi, x]) \Rightarrow
\text{cntVN } [\psi] \ \text{box } (\psi, x. U[\text{id}_\psi, x])
+ \text{cntVN } [\psi] \ \text{box } (\psi, x. V[\text{id}_\psi, x])$ 
```

The function `cntV` is built by a context abstraction $\Lambda \psi$ that introduces the context variable ψ and binds every occurrence of ψ in the body. Next, we introduce the computation-level variable f which has type $\text{o}[\psi, x:\text{nat}]$. In the body of the function `cntV` we case-analyze objects of type $\text{o}[\psi, x:\text{nat}]$. The `box` construct separates data from computations.

When we encounter an object built from a constructor `eq`, `imp`, or `forall`, we need to access the subexpression(s) underneath. Pattern variables are characterized by a closure $U[\sigma]$ consisting of a contextual variable U and a *postponed substitution* σ . As soon as we know what the contextual variable stands for, we apply the substitution σ . In the example, the postponed substitution associated with U is the identity substitution which essentially corresponds to α -renaming. We write id_ψ for the identity substitution with domain ψ . Intuitively, one may think of the substitution associated with contextual variables which occur in patterns as a list of variables which may occur in the hole. In the data object $U[\text{id}_\psi]$, for example, the contextual variable U

can be instantiated with any formula that is either closed (i.e. it does not refer to any bound variable listed in the context ψ) or contains a bound variable from the context ψ . Since we want to allow subformulas to refer to all variables in $(\psi, x:\text{nat})$, we write $U[\text{id}_\psi, x]$. We use capital letters for meta-variables.

In the first case for **imp** we recursively analyze the subformulas described by $U[\text{id}_\psi, x]$ and $W[\text{id}_\psi, x]$. The context ψ is the same, so we pass it (explicitly) in the recursive calls.

In the case for **box**($\psi, x. \text{forall } (\lambda y. W[\text{id}_\psi, x, y])$), we analyze the formula $W[\text{id}_\psi, x, y]$ under the assumption that y is a natural number. To do this, we pass an extended context $(\psi, y:\text{nat})$ to **cntV**.

Finally, for **eq**, we call **cntVN** to count the occurrences of x in the natural numbers $U[\text{id}_\psi, x]$ and $V[\text{id}_\psi, x]$, explicitly passing the context ψ . The function **cntVN** counts the occurrences of a variable x in an object of type $\text{nat}[\psi, x:\text{nat}]$.

```

rec cntVN :  $\Pi \psi:\text{natCtx}.\text{nat}[\psi, x:\text{nat}] \rightarrow \text{int} =$ 
 $\Lambda \psi \Rightarrow \text{fn } n \Rightarrow \text{case } n \text{ of}$ 
   $\text{box}(\psi, x. x) \Rightarrow 1$ 
 $| \text{box}(\psi, x. p[\text{id}_\psi]) \Rightarrow 0$ 
 $| \text{box}(\psi, x. \text{Zero}) \Rightarrow 0$ 
 $| \text{box}(\psi, x. \text{Suc } U[\text{id}_\psi, x]) \Rightarrow$ 
   $\text{cntVN } [\psi] \text{ box}(\psi, x. U[\text{id}_\psi, x])$ 

```

The first two cases are the interesting ones. The pattern $\text{box}(\psi, x. x)$ matches an occurrence of x . The second pattern, $\text{box}(\psi, x. p[\text{id}_\psi])$, matches a variable that is not x and occurs in ψ . For this case, we use a *parameter variable* p (the small letter distinguishes it from a meta-variable). This represents a bound data-level variable. The substitution id_ψ associated with p characterizes the possible instantiations of p . This allows us to write a generic base case where we encounter a parameter from the context that ψ stands for. This combination of explicit context variables and parameter variables to describe generic base cases is unique to our work. Comparison and explicit pattern matching for variables are features typically associated with nominal systems [7, 27]. However, unlike nominal systems, variable names here are not global, but local and subject to α -renaming.

In Twelf [19], one can write a relation that counts variable occurrences, but there is no generic base case for counting variables in a natural number. Instead, one dynamically adds the appropriate base case for every variable introduced when traversing a universal quantifier. Similarly, in Delphin [23], one cannot write such a generic base case, and must keep track of the appropriate result for each variable.

2.2 Example: Programming with proofs

We illustrate the idea of programming with proofs by considering bracket abstraction, a key lemma that arises in the translation of natural deduction proofs to Hilbert-style proofs. For this discussion we concentrate on the fragment consisting of implications. This example has been extensively discussed in the literature [18, 23] and so highlights the differences between approaches. This example again uses explicit context variables and parameter variables. The Hilbert-style axiomatization can be formalized as follows:

```

hil : o  $\rightarrow$  type.
k : hil (A imp (B imp A)).
s : hil ((A imp (B imp C)) imp
  ((A imp B) imp (A imp C))).
mp : hil (A imp B)  $\rightarrow$  hil A  $\rightarrow$  hil B.

```

We omit leading Π s from the types when they can be reconstructed, as is common practice in the Twelf system [19], an implementation of the logical framework LF.

The bracket abstraction lemma states that a derivation $\mathcal{D}$ of **hil** B that depends on the hypothesis $x:\text{hil } A$ can be translated into a derivation $\mathcal{E}$ of **hil** $(A \text{ imp } B)$ that does not refer to the hypothesis x :

LEMMA (BRACKET ABSTRACTION)

If $\mathcal{D} :: \Gamma, x:\text{hil } A \vdash \text{hil } B$ then $\mathcal{E} :: \Gamma \vdash \text{hil } (A \text{ imp } B)$

The context Γ has the form $x_n:\text{hil } A_n, \dots, x_1:\text{hil } A_1$, and the proof follows by induction on the derivation $\mathcal{D}$. There are four base cases. Two arise from the constants **k** and **s**, but we concentrate on the other two cases, which arise from using an assumption in $\Gamma, x:\text{hil } A$: we could have used the assumption $x:\text{hil } A$, or an assumption $x_i:A_i$ from Γ , to prove **hil** B .

Explicit contexts allow us to easily characterize the lemma as a dependent type, and implement the inductive proof as a recursive program using pattern matching. Moreover, every case in the informal proof directly corresponds to a case in our recursive program. As a first step, we characterize the context Γ with a context schema:

ctx schema **hilCtx** = (all $A:o. \text{hil } A$)*.

This schema describes contexts $x_n:\text{hil } A_n, \dots, x_1:\text{hil } A_1$. Next, we represent the lemma as a dependent type. The recursive function that realizes this type describes the proof transformation on Hilbert derivations. Implicit data objects that characterize index objects occurring in a dependent type are introduced with $\Pi^\square$, but we omit these implicit type arguments from computations and from data for readability, since we expect them to be reconstructed in practice.

```

rec ded :  $\Pi \gamma:\text{hilCtx}.\Pi^\square A:o[\gamma]. \Pi^\square B:o[\gamma].$ 
  (hil B[id $_\gamma$ ]) [ $\gamma, x:\text{hil } A[\text{id}_\gamma]$ ]
 $\rightarrow$  (hil (A[id $_\gamma$ ] imp B[id $_\gamma$ ])) [ $\gamma$ ] =
 $\Lambda \gamma \Rightarrow \text{fn } D \Rightarrow$ 
case D of
  box( $\gamma, x. k$ )  $\Rightarrow$  box( $\gamma. \text{mp } k k$ )
 $| \text{box}(\gamma, x. s) \Rightarrow \text{box}(\gamma. \text{mp } k s)$ 
 $| \text{box}(\gamma, x. x) \Rightarrow \text{box}(\gamma. \text{mp } (\text{mp } s k) k)$ 
 $| \text{box}(\gamma, x. p[\text{id}_\gamma]) \Rightarrow \text{box}(\gamma. \text{mp } k p[\text{id}_\gamma])$ 
 $| \text{box}(\gamma, x. \text{mp } D_1[\text{id}_\gamma, x] D_2[\text{id}_\gamma, x]) \Rightarrow$ 
  let
    box( $\gamma. E_1[\text{id}_\gamma]$ ) = ded [ $\gamma$ ] box( $\gamma, x. D_1[\text{id}_\gamma, x]$ )
    box( $\gamma. E_2[\text{id}_\gamma]$ ) = ded [ $\gamma$ ] box( $\gamma, x. D_2[\text{id}_\gamma, x]$ )
  in
    box( $\gamma. \text{mp } (\text{mp } s E_1[\text{id}_\gamma]) E_2[\text{id}_\gamma]$ )
end

```

We analyze Hilbert-style derivations by pattern matching on D , which describes an object of type **hil** $B[\text{id}_\gamma]$ in the context $(\gamma, x:\text{hil } A[\text{id}_\gamma])$. Intuitively, we can construct all derivations of the appropriate type by either using one of the constructors or an element from the context. This gives rise to the first two cases: either we have used the specific assumption x , or we have used one of the assumptions in γ . To describe this last generic variable case, we again use a parameter variable.

The other base cases are straightforward, so we concentrate on the last case where we use modus ponens. To pattern match on derivations constructed using the constant **mp**, we describe the subderivations using meta-variables $D_1[\text{id}_\gamma]$ and $D_2[\text{id}_\gamma]$ for which higher-order pattern matching will

find appropriate instantiations during runtime. The appeal to the induction hypothesis corresponds to the recursive call to the function `ded`. We first pass to this function the context γ by context application, and then the derivation described by `box( $\gamma, x, D_1[id_\gamma, x]$ )`.

2.3 Summary of key ideas

We summarize here the five key ideas underlying our work: First, we separate data from computations via the modality `box`. Second, every data object is closed with respect to a local context. For example, `box(x. eq x z)` denotes an object of type `o[x:nat]`, i.e. a proposition that may contain the variable x . The `box`-construct introduces the bound variables x . Third, we allow context variables ψ and abstract over them on the computation level. This is necessary since the concrete bound variables occurring in a data object are only exposed when we recursively traverse a binder, and the context describing these variables may grow. More importantly, this allows us to program with explicit contexts and capture more invariants. Fourth, we support pattern matching on higher-order data using meta-variables and, importantly, parameter variables. While meta-variables allow us to deconstruct arbitrary objects with binders, parameter variables allow us to manipulate names of bound variables directly in computation. Finally, we support dependent types using the type $\Pi^{\square} u::A[\Psi].\tau$, and we clearly distinguish implicit data objects occurring as index objects in data-level types from explicit data objects that can be analyzed by pattern matching.

3. DATA-LEVEL TERMS AND CONTEXTS

We begin by describing the data layer of our dependently-typed intermediate language. We support the full logical framework LF together with Σ -types. Our data layer closely follows contextual modal type theory [15], extended with parameter variables and context variables [21], and finally with dependent pairs and projections¹.

Kinds	K	$::=$	<code>type</code> $\Pi x:A.K$
Atomic types	P	$::=$	a $M_1 \dots M_n$
Types	A, B	$::=$	P $\Pi x:A.B$ $\Sigma x:A.B$
Normal terms	M, N	$::=$	$\lambda x.M$ (M, N) R
Neutral terms	R	$::=$	c x $u[\sigma]$ $p[\sigma]$ R $\text{proj}_k R$
Substitutions	σ, ρ	$::=$	$\cdot$ $\sigma ; M$ σ, R id_ψ

We only characterize normal terms since only these are meaningful in the logical framework, following Watkins et al. [28] and Nanevski et al. [15]. This is achieved by distinguishing between normal terms M and neutral terms R . While the syntax only guarantees that terms N contain no β -redexes, the typing rules will also guarantee that all well-typed terms are fully η -expanded.

We distinguish between four kinds of variables in our theory: *Ordinary bound variables* are used to represent data-level binders and are bound by λ -abstraction. *Contextual variables* stand for open objects, and include *meta-variables* u that represent general open objects and *parameter variables* p that can only be instantiated with an ordinary bound

variable. Contextual variables are introduced in computation-level case expressions, and can be instantiated via pattern matching. They are associated with a postponed substitution σ thereby representing a closure. This substitution precisely characterizes the dependencies we allow when instantiating the meta-variable with an open term. Our intention is to apply σ as soon as we know which term the contextual variable should stand for. The domain of σ thus describes the free variables of the object the contextual variable stands for, and the type system statically guarantees this.

Substitutions σ are built of either normal terms (in $\sigma ; M$) or atomic terms (in σ, R). The two forms are necessary since not every neutral term is also a normal term. Only neutral terms of atomic type are in fact normal. We do not make the domain of the substitutions explicit, which simplifies the theoretical development and avoids having to rename the domain of a given substitution σ . We also require a first-class notion of identity substitution id_ψ . Data-level substitutions, as defined operations on data-level terms, are written $[\sigma]N$.

While contextual variables are declared in a meta-level context Δ , ordinary bound variables are declared in a data-level context Ψ . Our foundation supports *context variables* ψ which allow us to reason abstractly with contexts. Context variables are declared in Ω . Unlike previous uses of context variables [12], a context may contain at most one context variable. In the same way that types classify objects, and kinds classify types, we introduce the notion of a schema W that classifies contexts Ψ .

Context variables	ψ, ϕ
Contexts	$\Psi, \Phi ::= \cdot$ ψ $\Psi, x:A$
Meta-contexts	$\Delta ::= \cdot$ $\Delta, u::A[\Psi]$ $\Delta, p::A[\Psi]$
Schema contexts	$\Omega ::= \cdot$ $\Omega, \psi::W$
Element types	$\tilde{A} ::= \Pi x:A.\tilde{A}$ a $N_1 \dots N_n$
Schema elements	$F ::= \text{all } x_1:\tilde{B}_1, \dots, x_n:\tilde{B}_n.$ $\Sigma y_1:\tilde{A}_1, \dots, y_k:\tilde{A}_k.\tilde{A}$
Schemas	$W ::= (F_1 + \dots + F_n)^*$

As the earlier example illustrated, contexts play an important part in programming with open data objects, and contexts, which are explicitly constructed at the computation level, belong to a specific context schema. For example, the schema `(nat)*` represented contexts of the form $x_1:\text{nat}, \dots, x_n:\text{nat}$, and the schema `all A:o. hil A` characterized the context Γ consisting of assumptions $x:\text{hil } A$. In general, we allow an even richer form of contexts to be captured. Consider how the schema of the context Γ will change when we extend the Hilbert-style calculus with universal quantifiers. In this case, we must also consider assumptions about possible parameters introduced when we traverse a universal quantifier. On paper, we may write the following to characterize this context:

$$\text{Hilbert Contexts } \Gamma ::= \cdot$$
 | $\Gamma, a:\text{nat}$ | $\Gamma, x:\text{hil } A$

We provide $+$ to denote a choice of possible elements in a context. Hence we would modify our schema for Hilbert contexts as follows:

$$\text{ctx schema hilCtx} = \text{nat} + \text{all } A:o. \text{hil } A.$$

Context schemas are built of elements $F_1, \dots, F_n$, each of which must have the form $\text{all } \tilde{\Phi}.\Sigma \tilde{\Psi}.\tilde{A}$ where $\tilde{\Phi}$ and $\tilde{\Psi}$ are

¹Prior work also considered substitution variables. Although our theory extends to substitution variables, we omit them here to make the presentation compact and focus on aspects related to dependent types.

Σ -free contexts, i.e. $x_1:\widetilde{B}_1, \dots, x_k:\widetilde{B}_k$. In other words, the element is of $\Sigma\Pi$ -type, where we first introduce some Σ -types, followed by pure Π -types. We disallow arbitrary mixing of Σ and Π . This restriction makes it easier to describe the possible terms of this type, which is a crucial step towards ensuring coverage [6].

Schemas resemble Schürmann's worlds [24], but while similar in spirit, we draw on the power of dependent pairs to express the relationship between multiple objects in a context. While worlds impose a similar $\Sigma\Pi$ -structure, schemas differ from worlds in the sense that schemas are pure. They only keep track of assumptions about binders occurring in a data object of type A . This is unlike worlds as realized in Twelf [19], where world declarations also track dynamic computation-level extensions.

3.1 Data-level typing

In this section, we present a bidirectional type system for data-level terms. We assume that type constants and object constants are declared in a signature Σ , which we suppress since it is the same throughout a typing derivation. However, we will keep in mind that all typing judgments have access to a well-formed signature. Typing is defined via the following judgments:

$$\begin{array}{ll} \Omega; \Delta; \Psi \vdash M \Leftarrow A & \text{Check normal object } M \text{ against } A \\ \Omega; \Delta; \Psi \vdash R \Rightarrow A & \text{Synthesize } A \text{ for neutral object } R \\ \Omega; \Delta; \Phi \vdash \sigma \Leftarrow \Psi & \text{Check } \sigma \text{ against context } \Psi \end{array}$$

For readability, we omit Ω in the subsequent development since it is constant; we also assume that Δ and Ψ are well-formed. First, the typing rules for objects.

Data-level normal terms

$$\begin{array}{c} \frac{\Delta; \Psi, x:A \vdash M \Leftarrow B}{\Delta; \Psi \vdash \lambda x. M \Leftarrow \Pi x:A. B} \text{ III} \\ \frac{\Delta; \Psi \vdash M_1 \Leftarrow A_1 \quad \Delta; \Psi \vdash M_2 \Leftarrow [M_1/x]_{A_1}^a A_2}{\Delta; \Psi \vdash (M_1, M_2) \Leftarrow \Sigma x:A_1. A_2} \text{ SII} \\ \frac{\Delta; \Psi \vdash R \Rightarrow P' \quad P' = P}{\Delta; \Psi \vdash R \Leftarrow P} \text{ turn} \end{array}$$

Data-level neutral terms

$$\begin{array}{c} \frac{x:A \in \Psi}{\Delta; \Psi \vdash x \Rightarrow A} \text{ var} \quad \frac{c:A \in \Sigma}{\Delta; \Psi \vdash c \Rightarrow A} \text{ con} \\ \frac{u::A[\Phi] \in \Delta \quad \Delta; \Psi \vdash \sigma \Leftarrow \Phi}{\Delta; \Psi \vdash u[\sigma] \Rightarrow [\sigma]_{\Phi}^a A} \text{ mvar} \\ \frac{p::A[\Phi] \in \Delta \quad \Delta; \Psi \vdash \sigma \Leftarrow \Phi}{\Delta; \Psi \vdash p[\sigma] \Rightarrow [\sigma]_{\Phi}^a A} \text{ param} \\ \frac{\Delta; \Psi \vdash R \Rightarrow \Pi x:A. B \quad \Delta; \Psi \vdash N \Leftarrow A}{\Delta; \Psi \vdash R N \Rightarrow [N/x]_A^a B} \text{ IIIE} \\ \frac{\Delta; \Psi \vdash R \Rightarrow \Sigma x:A_1. A_2}{\Delta; \Psi \vdash \text{proj}_1 R \Rightarrow A_1} \text{ SIE}_1 \\ \frac{\Delta; \Psi \vdash R \Rightarrow \Sigma x:A_1. A_2}{\Delta; \Psi \vdash \text{proj}_2 R \Rightarrow [\text{proj}_1 R/x]_{A_1}^a A_2} \text{ SIE}_2 \end{array}$$

Data-level substitutions

$$\begin{array}{c} \frac{}{\Delta; \Psi \vdash \cdot \Leftarrow \cdot} \quad \frac{}{\Delta; \psi, \Psi \vdash \text{id}_{\psi} \Leftarrow \psi} \\ \frac{\Delta; \Psi \vdash \sigma \Leftarrow \Phi \quad \Delta; \Psi \vdash R \Rightarrow A' \quad [\sigma]_{\Phi}^a A = A'}{\Delta; \Psi \vdash (\sigma, R) \Leftarrow (\Phi, x:A)} \\ \frac{\Delta; \Psi \vdash \sigma \Leftarrow \Phi \quad \Delta; \Psi \vdash M \Leftarrow [\sigma]_{\Phi}^a A}{\Delta; \Psi \vdash (\sigma, M) \Leftarrow (\Phi, x:A)} \end{array}$$

We assume that data level type constants a together with constants c have been declared in a signature. We will tacitly rename bound variables, and maintain that contexts and substitutions declare no variable more than once. Note that substitutions σ are defined only on ordinary variables x , not on modal variables u . We also require the usual conditions on bound variables. For example, in the rule for lambda-abstraction the bound variable x must be new and cannot already occur in the context Ψ . This can always be achieved via α -renaming. The typing rules for data-level neutral terms rely on *hereditary substitutions* which ensure that canonical forms are preserved [28, 15].

The idea is to define a primitive recursive functional that always returns a canonical object. In places where the ordinary substitution would construct a redex $(\lambda y. M) N$ we must continue, substituting N for y in M . Since this could again create a redex, we must continue and hereditarily substitute and eliminate potential redexes. Hereditary substitution can be defined recursively, considering both the structure of the term to which the substitution is applied and the type of the object being substituted. Hence we annotate the substitution with the type of the argument being substituted. We also indicate with the superscript a that the substitution is applied to a type. We give the definition of hereditary substitutions in the appendix.

Finally, we define context checking.

Context Ψ checks against a schema $W : \Omega \vdash \Psi \Leftarrow W$

for some k

$$\frac{\Omega; \Delta; \Psi \vdash B \in F_k \quad \Omega; \Delta \vdash \Psi \Leftarrow (F_1 + \dots + F_n)^*}{\Omega; \Delta \vdash \Psi, x:B \Leftarrow (F_1 + \dots + F_n)^*}$$

$$\frac{\psi::W \in \Omega}{\Omega; \Delta \vdash \psi \Leftarrow W} \quad \frac{}{\Omega; \Delta \vdash \cdot \Leftarrow W}$$

To verify that a context Ψ belongs to a schema $W = (F_1 + \dots + F_n)^*$, we check that for every declaration B_i in the context Ψ , there exists a k s.t. B_i is an instance of a schema element $F_k = \text{all } \tilde{\Phi}. \Sigma y_1:\tilde{A}_1, \dots, y_j:\tilde{A}_j. \tilde{A}$. This means we must find an instantiation σ for all the variables in $\tilde{\Phi}$ s.t. $[\sigma]_{\tilde{\Phi}}^a (\Sigma y_1:\tilde{A}_1, \dots, y_j:\tilde{A}_j. \tilde{A})$ is equal to B_i . This is done in practice by higher-order pattern matching.

THEOREM 1 (DECIDABILITY OF TYPECHECKING).

All judgments in the contextual modal type theory are decidable.

PROOF. The typing judgments are syntax-directed and therefore clearly decidable assuming hereditary substitution is decidable. $\square$

3.2 Substitution operations

The different variables (ordinary variables x , context variables ψ , and contextual variables $u[\sigma]$ and $p[\sigma]$) give rise to

different substitution operations. We already remarked on the hereditary substitution operation for ordinary variables x and we give its definition in the appendix. The remaining substitution operations do not require any significant changes from earlier work [21, 15] to handle dependent types and we revisit them in this section.

Substitution for context variables

If we encounter a context variable ψ , we simply replace it with the context Ψ .

Data-level context

$$\begin{aligned} \llbracket \Psi/\psi \rrbracket(\cdot) &= \cdot \\ \llbracket \Psi/\psi \rrbracket(\Phi, x:A) &= (\Phi', x:A') \quad \text{if } x \notin V(\Phi') \text{ and } \llbracket \Psi/\psi \rrbracket A = A' \\ &\quad \text{and } \llbracket \Psi/\psi \rrbracket \Phi = \Phi' \\ \llbracket \Psi/\psi \rrbracket(\psi) &= \Psi \\ \llbracket \Psi/\psi \rrbracket(\phi) &= \phi \quad \text{if } \phi \neq \psi \end{aligned}$$

When we apply the substitution $\llbracket \Psi/\psi \rrbracket$ to the context $\Phi, x:A$, we apply the substitution to the type A , yielding some new type A' , and to the context Φ , yielding some new context Φ' . Applying the substitution to the type A is necessary in the dependently-typed setting, since A may contain terms and in particular identity substitutions id_ψ . When we replace ψ with Ψ in the substitution id_ψ , we unfold the identity substitution. Expansion of the identity substitution is defined by the operation $\text{id}(\Psi)$ for valid contexts Ψ :

$$\begin{aligned} \text{id}(\cdot) &= \cdot \\ \text{id}(\Psi, x:A) &= \text{id}(\Psi), x \\ \text{id}(\psi) &= \text{id}_\psi \end{aligned}$$

LEMMA 1 (UNFOLDING IDENTITY SUBSTITUTION).
If $\text{id}(\Psi) = \sigma$ then $\Delta; \Psi, \Psi' \vdash \sigma \Leftarrow \Psi$.

PROOF. By induction on the structure of Ψ . $\square$

When we combine Φ' and the declaration $x:A'$ to yield a new context, we must ensure that x is not already declared in Φ' . This can always be achieved by appropriately renaming bound variable occurrences. We write $V(\Phi')$ for the set of variables declared in Φ' . The rest of the definition is mostly straightforward.

THEOREM 2 (SUBSTITUTION FOR CONTEXT VARIABLES).
If $\Omega, \psi::W, \Omega'; \Delta; \Phi \vdash J$ and $\Omega \vdash \Psi \Leftarrow W$
then $\Omega, \Omega'; \llbracket \Psi/\psi \rrbracket \Delta; \llbracket \Psi/\psi \rrbracket(\Phi) \vdash \llbracket \Psi/\psi \rrbracket J$.

PROOF. By induction on the first derivation using Lemma 1. $\square$

Contextual substitution for contextual variables

Substitution for contextual variables is a little more difficult, but is essentially similar to the definitions in [21]. We can think of $u[\sigma]$ as a closure where, as soon as we know which term u should stand for, we can apply σ to it. The typing will ensure that the type of M and the type of u agree, i.e. we can replace u of type $A[\Psi]$ with a normal term M if M has type A in the context Ψ . Because of α -conversion, the variables substituted at different occurrences of u may differ, and we write the contextual substitution as $\llbracket \hat{\Psi}.M/u \rrbracket(N)$, $\llbracket \hat{\Psi}.M/u \rrbracket(R)$, and $\llbracket \hat{\Psi}.M/u \rrbracket(\sigma)$, where $\hat{\Psi}$ binds all free variables in M . Applying $\llbracket \hat{\Psi}.M/u \rrbracket$ to the closure $u[\sigma]$ first obtains the simultaneous substitution $\sigma' = \llbracket \hat{\Psi}.M/u \rrbracket \sigma$, but instead of returning $M[\sigma']$, it eagerly applies σ' to M . Similar ideas apply to parameter substitutions, which are written

$\llbracket \hat{\Psi}.x/p \rrbracket(M)$, $\llbracket \hat{\Psi}.x/p \rrbracket(R)$ and $\llbracket \hat{\Psi}.x/p \rrbracket(\sigma)$. Parameter substitution could not be achieved with the previous definition of contextual substitution for meta-variables, since it only allows us to substitute a normal term for a meta-variable, and x is only normal if it is of atomic type. We give its definition in the appendix.

Finally, we will rely in the subsequent development on simultaneous contextual substitutions, built of either meta-variables, $(\theta, \hat{\Psi}.M/u)$, or parameter variables, $(\theta, \hat{\Psi}.x/p)$. The judgment $\Delta \vdash \theta \Leftarrow \Delta'$ checks that the contextual substitution θ maps contextual variables from Δ' to the contextual variables in Δ .

Simultaneous contextual substitution

$$\begin{aligned} \frac{}{\Delta \vdash \cdot \Leftarrow \cdot} \quad & \frac{\Delta \vdash \theta \Leftarrow \Delta' \quad \Delta; \llbracket \theta \rrbracket \Psi \vdash M \Leftarrow \llbracket \theta \rrbracket A}{\Delta \vdash (\theta, \hat{\Psi}.M/u) \Leftarrow \Delta', u::A[\Psi]} \\ \frac{\Delta \vdash \theta \Leftarrow \Delta' \quad \Delta; \llbracket \theta \rrbracket \Psi \vdash x \Rightarrow A' \quad A' = \llbracket \theta \rrbracket A}{\Delta \vdash (\theta, \hat{\Psi}.x/p) \Leftarrow \Delta', p::A[\Psi]} \end{aligned}$$

4. COMPUTATION-LEVEL EXPRESSIONS

We cleanly separate the data level from the computation level, which describes programs operating on data. Computations analyze data of type $A[\Psi]$, which denotes an object of type A that may contain the variables specified in Ψ . To allow quantification over context variables ψ , we introduce the type $\Pi\psi:W.\tau$ and context abstraction $\Lambda\psi.e$. We write $\rightarrow$ for computation-level functions.

As mentioned earlier, the language we describe in this section may be thought of as an internal language into which source level programs will be compiled. The extension to dependent types is novel in that we clearly distinguish between (1) *implicit data objects* which occur in a type as index and are kept implicit in the source program and (2) *explicit data objects* which are analyzed recursively by pattern matching. The intuition is that implicit arguments can be reconstructed when translating source programs to our intermediate representation. We introduce abstraction over implicit data objects occurring in a dependent type A on the computation level using the dependent type $\Pi^\square u::A[\Psi].\tau$. Moreover, we require that patterns occurring in the branches of case expressions are annotated with types, since the type of each pattern need not be identical to the type of the expression being case analyzed in the dependently-typed setting. We expect that these annotations can be reconstructed during the compilation from source programs to this intermediate language.

Types	$\tau ::= A[\Psi] \mid \tau_1 \rightarrow \tau_2 \mid \Pi\psi::W.\tau \mid \Pi^\square u::A[\Psi].\tau$
Expressions	$e ::= i \mid \text{rec } f.e \mid \text{fn } y.e \mid \Lambda\psi.e \mid \lambda^\square u.e$
(checked)	$\mid \text{box}(\hat{\Psi}.M) \mid \text{case } i \text{ of } bs$
Expressions	$i ::= y \mid i \ e \mid i \ [\Psi] \mid i \ [\hat{\Psi}.M] \mid (e : \tau)$
(synth.)	
Branch	$b ::= \Pi\Delta.\text{box}(\hat{\Psi}.M) : A[\Psi] \mapsto e$
Branches	$bs ::= \cdot \mid (b \mid bs)$
Contexts	$\Gamma ::= \cdot \mid \Gamma, y:\tau$

We will enforce that all context variables are bound by Λ -abstractions. To support α -renaming of ordinary bound variables, we write $\text{box}(\hat{\Psi}.M)$ where $\hat{\Psi}$ is a list of variables that can possibly occur in M . Index objects of dependent types are introduced by $\lambda^\square u.e$ of type $\Pi^\square u::A[\Psi].\tau$. In other

words, index objects are characterized by contextual variables. $i \Vdash M$ describes the application of an index argument to an expression. The contextual variables in branches b , declared in Δ , are instantiated using higher-order pattern matching. We only consider patterns à la Miller [13] where meta-variables that are subject to instantiation must be applied to a distinct set of bound variables. In our setting, this means all contextual variables must be associated with a substitution such as $x_{\Phi(1)}/x_1, \dots, x_{\Phi(n)}/x_n$. This fragment is decidable and has efficient algorithms for pattern matching [20, 22].

Patterns in case expressions are annotated with their types, since in the dependently-typed setting, the type of each pattern need not be identical to the type of the expression being analyzed.

4.1 Computation-level typing

We describe computation-level typing using the following judgments:

$$\begin{array}{ll} \Omega; \Delta; \Gamma \vdash e \Leftarrow \tau & e \text{ checks against } \tau \\ \Omega; \Delta; \Gamma \vdash i \Rightarrow \tau & i \text{ synthesizes } \tau \\ \Omega; \Delta; \Gamma \vdash b \Leftarrow_{\tau'} \tau & \text{branch } b \text{ checks against } \tau, \\ & \text{when case-analyzing a } \tau' \end{array}$$

The rules are given in Figure 1. The names of computation-level rules are written with a line above to distinguish them from the names of data-level rules, which are underlined. There are two interesting rules. The first one is $\overline{\text{turn}}$. In this rule we check that the computation-level types τ and τ' are equal. At the data level, we only characterize canonical forms, so equality between two dependent types A and A' is simply syntactic equality. Hence, equality between two computation-level types is also just syntactic equality. This is in stark contrast to dependently-typed languages such as Cayenne [1] and Epigram [11] that allow computations within the index objects of dependent types. In these systems, we cannot simply compare two types syntactically, but must evaluate the index arguments first, before comparing the final values. Even weaker dependently-typed systems such as DML [30], where types are indexed by a decidable constraint domain such as integers with linear inequalities, need efficient constraint solvers to decide type equality.

The second interesting rule in the bidirectional type checking algorithm is the one for branches, since the type $A_k[\Psi_k]$ of each of the patterns must be considered equal to the type $A[\Psi]$, the type of the expression we analyze by cases. In some approaches to dependently typed programming [30], branches are checked under certain equality constraints. Instead we propose here to solve these constraints eagerly using higher-order pattern unification [13]. Hence, we restrict the contextual variables $u[\sigma]$ that occur in patterns to be higher-order patterns, i.e. the substitution σ is simply a renaming substitution. Higher-order pattern unification is decidable, so unification of the contexts and types is decidable. For a formal description of a higher-order pattern unification algorithm for contextual variables, see [20].

We use the judgment $\Omega; \Delta \vdash A \doteq A_k / (\theta, \Delta')$ to describe higher-order pattern unification for types. Δ describes all the meta-variables occurring in types A and A_k . The result of unifying A and A_k is a contextual substitution θ that maps the meta-variables in Δ to the meta-variables in Δ' s.t. $\llbracket \theta \rrbracket A$ is equal to $\llbracket \theta \rrbracket A_k$. This operation can be extended to unify contexts Ψ and Ψ_k .

Unlike the simply-typed setting, where it is natural to require linearity, i.e. that every contextual variable in a pattern occurs only once, we cannot enforce a similar condition in the dependently-typed setting, because the object may become ill-typed. To illustrate, consider pattern matching against the Hilbert derivation $\text{box}(\psi. \text{mp } k \ D[\text{id}_\psi])$. After reconstructing the omitted implicit arguments, we get $\text{box}(\psi. \text{mp } A[\text{id}_\psi] \ (A[\text{id}_\psi] \ \text{imp } B[\text{id}_\psi]) \ k \ D[\text{id}_\psi])$. It is clear that enforcing linearity in dependently-typed patterns is impossible.

Let us now return to the rule for branches. Branches b have the form $\Pi \Delta. \text{box}(\hat{\Psi}. M) \mapsto e$ where Δ contains all the contextual variables introduced and occurring in the guard $\text{box}(\hat{\Psi}. M)$. We concentrate here on the last rule for checking the pattern in a case expression. After typing the pattern in the case expression, we unify the type of the subject of the case expression with the type of the pattern. First, however, we must unify the context Ψ with the context Ψ_i of the pattern. Finally, we apply the result of higher-order unification θ to the body of the branch and check that it is well-typed.

This approach will simplify the preservation and progress proof. It is also closer to a realistic implementation, which would support early failure and indicate the offending branch.

THEOREM 3 (DECIDABILITY OF TYPECHECKING).

Computation-level typechecking is decidable.

PROOF. The typing rules are syntax-directed. Computation-level types are in canonical form, so the equality in rule $\overline{\text{turn}}$ is syntactic equality. Moreover, higher-order pattern matching is decidable. Thus, typechecking is decidable. $\square$

Finally, we briefly remark on computation-level substitutions. There are four varieties: $[e/x]e'$ describes the substitution of the computation-level expression e for x in the computation-level expression e' . This operation is straightforward and capture-avoiding in the case of functions $\text{fn } y.e$. It does not affect data-level terms, since the computation-level variable x cannot occur in them.

$\llbracket \Psi/\psi \rrbracket(e)$ extends the previous definition of context substitution to the computation level. It is capture-avoiding in the case for context abstraction, and is propagated to the data level.

Finally, $\llbracket \theta \rrbracket(e)$ and $\llbracket \hat{\Psi}. M/u \rrbracket(e)$ extend the previous contextual substitution operation to the computation level in a straightforward manner. We ensure it is capture-avoiding in $\lambda^\square$ -abstraction and in branches of case-expressions.

4.2 Operational semantics

Next, we define a small-step evaluation judgment:

$$e \text{ evaluates in one step to } e' \quad e \longrightarrow e'$$

$$\begin{array}{l} \text{Branch } b \text{ matches } \text{box}(\hat{\Psi}. M) \text{ and steps to } e' \\ (\text{box}(\hat{\Psi}. M) : A[\Psi] \doteq b) \longrightarrow e' \end{array}$$

In the presence of full LF we cannot erase type information completely during evaluation, since not all implicit type arguments can be uniquely determined. Because evaluation relies on pattern matching, and higher-order pattern matching can only match two terms if they have the same type, we match against the pattern's type before matching against the pattern itself. Thus, we must translate the computational language into one where type annotations of the form $(e : \tau)$ are erased but all expressions of type $A[\Psi]$, in particular patterns in branches, carry their corresponding type.

Expression e checks against type τ

$$\begin{array}{c}
\frac{\Omega; \Delta; \Gamma, f: \tau \vdash e \Leftarrow \tau}{\Omega; \Delta; \Gamma \vdash \text{rec } f.e \Leftarrow \tau} \text{rec} \quad \frac{\Omega, \psi: W; \Delta; \Gamma \vdash e \Leftarrow \tau}{\Omega; \Delta; \Gamma \vdash \Lambda \psi.e \Leftarrow \Pi \psi: W. \tau} \overline{\Pi} \quad \frac{\Omega; \Delta; \Gamma, y: \tau_1 \vdash e \Leftarrow \tau_2}{\Omega; \Delta; \Gamma \vdash \text{fn } y.e \Leftarrow \tau_1 \rightarrow \tau_2} \overline{\rightarrow} \\
\frac{\Omega; \Delta, u::A[\Psi]; \Gamma \vdash e \Leftarrow \tau}{\Omega; \Delta; \Gamma \vdash \lambda^\square u. e \Leftarrow \Pi^\square u::A[\Psi]. \tau} \overline{\Pi^\square} \quad \frac{\Omega; \Delta; \Psi \vdash M \Leftarrow A}{\Omega; \Delta; \Gamma \vdash \text{box}(\hat{\Psi}. M) \Leftarrow A[\Psi]} \overline{\text{box}} \\
\frac{\Omega; \Delta; \Gamma \vdash i \Rightarrow A[\Psi] \quad \text{for all } k \ \Omega; \Delta; \Gamma \vdash b_k \Leftarrow A[\Psi]. \tau}{\Omega; \Delta; \Gamma \vdash \text{case } i \text{ of } b_1 \mid \dots \mid b_n \Leftarrow \tau} \overline{\text{case}} \quad \frac{\Delta; \Gamma \vdash i \Rightarrow \tau' \quad \Omega; \Delta \vdash \tau' = \tau}{\Omega; \Delta; \Gamma \vdash i \Leftarrow \tau} \overline{\text{turn}}
\end{array}$$

Expression i synthesizes type τ

$$\begin{array}{c}
\frac{\Omega; \Delta; \Gamma \vdash e \Leftarrow \tau}{\Omega; \Delta; \Gamma \vdash (e : \tau) \Rightarrow \tau} \overline{\text{anno}} \quad \frac{y: \tau \in \Gamma}{\Omega; \Delta; \Gamma \vdash y \Rightarrow \tau} \overline{\text{var}} \quad \frac{\Omega; \Delta; \Gamma \vdash i \Rightarrow \tau_2 \rightarrow \tau \quad \Omega; \Delta; \Gamma \vdash e \Leftarrow \tau_2}{\Omega; \Delta; \Gamma \vdash i e \Rightarrow \tau} \overline{\rightarrow E} \\
\frac{\Omega; \Delta; \Gamma \vdash i \Rightarrow \Pi \psi: W. \tau \quad \Omega; \Delta \vdash \Psi \Leftarrow W}{\Omega; \Delta; \Gamma \vdash i [\Psi] \Rightarrow [\Psi/\psi] \tau} \overline{\Pi E} \quad \frac{\Omega; \Delta; \Gamma \vdash i \Rightarrow \Pi^\square u::A[\Psi]. \tau \quad \Omega; \Delta; \Psi \vdash M \Leftarrow A}{\Omega; \Delta; \Gamma \vdash i [\hat{\Psi}. M] \Rightarrow [\hat{\Psi}. M/u] \tau} \overline{\Pi^\square E}
\end{array}$$

Body e_k checks against type τ , assuming the value cased upon has type $A[\Psi]$

$$\frac{\Omega; \Delta_k; \Psi_k \vdash M_k \Leftarrow A_k \quad \begin{array}{c} \Omega; \Delta, \Delta_k \vdash \Psi \doteq \Psi_k / (\theta_1, \Delta') \\ \Omega; \Delta' \vdash [\theta_1] A \doteq [\theta_1] A_k / (\theta_2, \Delta'') \\ \Omega; \Delta''; [\theta_2][\theta_1] \Gamma \vdash [\theta_2][\theta_1] e_k \Leftarrow [\theta_2][\theta_1] \tau \end{array}}{\Omega; \Delta; \Gamma \vdash \Pi \Delta_k. \text{box}(\hat{\Psi}. M_k) : A_k[\Psi_k] \mapsto e_k \Leftarrow A[\Psi] \tau}$$

Figure 1: Computation-level typing rules

We denote this translation by $|e|$ and $|i|$ for checked and synthesizing expressions, respectively².

Otherwise, the semantics is straightforward. In function application, values for program variables are propagated by computation-level substitution. Instantiations for context variables are propagated by applying a concrete context Ψ to a context abstraction $\Lambda \psi. e$. Index arguments are propagated in $(\lambda^\square u. e) [\hat{\Psi}. M]$ by replacing u with the concrete data object $\hat{\Psi}. M$.

Evaluation in branches relies on higher-order pattern matching against data-level terms to instantiate the contextual variables occurring in a branch. Data-level instantiations are propagated via simultaneous contextual substitution. We write $\text{box}(\hat{\Psi}. M) : A[\Psi] \Rightarrow \Leftarrow \Pi \Delta_1. \text{box}(\hat{\Psi}. M_1) : A_1[\Psi_1]$ to mean that higher-order pattern matching between $\text{box}(\hat{\Psi}. M_1)$ and $\text{box}(\hat{\Psi}. M)$ failed, i.e. there exists no instantiation for the contextual variables in Δ_1 that makes these terms equal.

We assume that $\text{box}(\hat{\Psi}. M)$ does not contain any meta-variables, i.e. it is closed, and that its type $A[\Psi]$ is known. Because of dependent types in Ψ_k we must first match Ψ against Ψ_k , and then proceed to match M against M_k .

Before we prove type safety for our dependently-typed functional language with higher-order abstract syntax, we briefly discuss the issue of coverage. To prove progress, we must validate that patterns in case expressions are exhaustive, i.e. that all possible cases are covered. In the first-order setting this is typically straightforward. Given a datatype

nat of natural numbers with constructors **Zero** and **Suc**, the set of patterns $Z = \{\text{Zero}, \text{Suc } u\}$ covers the type **nat**. However, in the higher-order setting we have open terms that can depend on assumptions. Intuitively, given a type $A[\Psi]$ we can generate a set of patterns by generating patterns for type A and in addition elements from Ψ . To generate all possible elements covering Ψ , we must inspect its shape. If $\Psi = \psi, x_1:A_1, \dots, x_n:A_n$, then we generate cases for $x_1, \dots, x_n$ along with a general parameter case, $p[\text{id}_\psi]$. For example, given a type **nat** $[\psi, x:\text{nat}]$ the set of patterns $Z = \{p[\text{id}_\psi], x, \text{Zero}, \text{Suc } u[\text{id}_\psi]\}$ covers all elements of this type. For a detailed discussion and soundness proof of coverage for higher-order data of type $A[\Psi]$, see [6]. Here, we simply assume that patterns cover all cases. First we state and prove a canonical forms lemma.

LEMMA 2 (CANONICAL FORMS).

- (1) If i is a value and $;; \vdash i \Rightarrow \tau \rightarrow \tau'$ then $|i| = \text{fn } y. |e'|$ and $;; y: \tau \vdash e' \Leftarrow \tau'$.
- (2) If i is a value and $;; \vdash i \Rightarrow A[\Psi]$ then $|i| = (\text{box}(\hat{\Psi}. M) : A[\Psi])$ and $;; \vdash \text{box}(\hat{\Psi}. M) \Leftarrow A[\Psi]$.
- (3) If i is a value and $;; \vdash i \Rightarrow \Pi^\square u::A[\Psi]. \tau$ then $|i| = \lambda^\square u. |e'|$ and $;; u::A[\Psi]; \vdash e' \Leftarrow \tau$.

PROOF. By induction on the typing derivation. $\square$

THEOREM 4 (PRESERVATION AND PROGRESS).

- (1) If $;; \vdash e \Leftarrow \tau$ and e coverage checks then either e is a value or there exists e' such that $|e| \rightarrow |e'|$ and $;; \vdash e' \Leftarrow \tau$.
- (2) If $;; \vdash i \Rightarrow \tau$ and i coverage checks then either i is a value or there exists i' such that $|i| \rightarrow |i'|$ and $;; \vdash i' \Rightarrow \tau$.

²Technically, this erasure is type-directed and annotates also the object $\hat{\Psi}. M$ in $i [\hat{\Psi}. M]$ with its corresponding type. This is necessary since contextual substitution is defined recursively on the structure of this type (see definition of contextual substitution in the appendix). For conciseness we omit this type here.

Evaluation of computations:

$$\begin{array}{c}
\frac{}{\text{rec } f.e \longrightarrow [\text{rec } f.e/f]e} \quad \frac{}{(\text{fn } y.e) v \longrightarrow [v/y]e} \quad \frac{}{(\lambda^{\square} u.e) [\hat{\Psi}.M] \longrightarrow [\hat{\Psi}.M/u]^e} \\
\frac{i_1 \longrightarrow i'_1}{i_1 e_2 \longrightarrow i'_1 e_2} \quad \frac{e_2 \longrightarrow e'_2}{v e_2 \longrightarrow v e'_2} \quad \frac{i \longrightarrow i'}{i [\Psi] \longrightarrow i' [\Psi]} \quad \frac{}{(\Lambda \psi.e) [\Psi] \longrightarrow [\Psi/\psi]e} \quad \frac{i \longrightarrow i'}{\text{case } i \text{ of } bs \longrightarrow \text{case } i' \text{ of } bs} \\
\frac{(\text{box}(\hat{\Psi}.M):A[\Psi]) \Rightarrow \Leftarrow (\Pi \Delta_1.\text{box}(\hat{\Psi}.M_1):A_1[\Psi_1]) \quad b = \Pi \Delta_1.\text{box}(\hat{\Psi}.M_1):A_1[\Psi_1] \mapsto e_1}{(\text{case } (\text{box}(\hat{\Psi}.M):A[\Psi]) \text{ of } b \mid bs) \longrightarrow \text{case } (\text{box}(\hat{\Psi}.M):A[\Psi]) \text{ of } bs} \\
\frac{(\text{box}(\hat{\Psi}.M):A[\Psi]) \doteq (\Pi \Delta.\text{box}(\hat{\Psi}.M_1):A_1[\Psi_1]) / \theta \quad b = \Pi \Delta.\text{box}(\hat{\Psi}.M_1):A_1[\Psi_1] \mapsto e_1}{(\text{case } (\text{box}(\hat{\Psi}.M):A[\Psi]) \text{ of } b \mid bs) \longrightarrow [\theta]e_1}
\end{array}$$

Evaluation of branch:

$$\frac{\Delta \vdash \Psi_k \doteq \Psi / (\theta_1, \Delta_1) \quad \Delta_1; \Psi \vdash A \doteq [\theta_1]A_k / (\theta_2, \Delta_2) \quad \theta = [\theta_2]\theta_1 \quad \Delta_2; \Psi \vdash M \doteq [\theta]M_k / (\theta_3, \cdot)}{(\text{box}(\hat{\Psi}.M) : A[\Psi] \doteq \Pi \Delta.\text{box}(\hat{\Psi}.M_k) : A_k[\Psi_k]) / [\theta_3]\theta}$$

Figure 2: Operational semantics

PROOF. By induction on the given typing derivation, using Lemma 2, the obvious substitution properties, and coverage [6] as needed. $\square$

5. RELATED WORK

Implementing proofs about HOAS encodings is an ambitious project. One approach is realized in Twelf [19], an implementation of the logical framework LF where inductive proofs are implemented as relations between derivations and the fact that relations constitute total functions is verified separately. While the logical framework LF itself is well-understood and has a small type-theoretic core, the external checkers guaranteeing totality of the implemented relations still remain mysterious to many users. Moreover, how to automate induction proofs about LF signatures and develop such proofs interactively has remained a major problem despite the seminal groundwork laid in Schürmann’s dissertation [24]. An alternative approach to proving properties about HOAS encodings is based on generic judgments [8] and realized in the system Abella. This approach enhances intuitionistic logic with generic judgments, which allow for recursive definitions and induction over natural numbers. Contexts are explicit, and the user needs to explicitly manage and prove properties about contexts. This is a powerful approach grounded in proof theory. In contrast, we aim for a type-theoretic approach for programming with proofs and explicit contexts. In particular, we characterize contexts type-theoretically using context schemas. A potential advantage of our approach is that we use the same functional paradigm of writing programs and writing proofs, and hence they can live within the same language.

Enriching functional programming with dependently-typed higher-order data structures is a longstanding open problem and is presently receiving widespread attention. Most closely related to our work is Delphin [23], a dependently-typed functional programming language that supports HOAS encodings. However, our theoretical approach differs substantially. Most of these differences arise because we build our type-theoretical foundation on contextual modal types where $A[\Psi]$ denotes an object M of type A in a context

Ψ . This means that the object M can refer to the variables declared in Ψ . Every data object is therefore locally closed. Moreover, the context Ψ is explicit. An important consequence is that, when pattern matching against data objects of type $A[\Psi]$, it is easy to understand intuitively when all cases are covered. A set of patterns is exhaustive if it contains all constructors of type A together with all the variables from Ψ . Furthermore, the power of context variables allows us to capture more general invariants about our programs. In Delphin, contexts are implicit and there are no context variables. This has several important consequences. For example, one cannot write out a generic case for parameters in ψ —as in Twelf, these generic base cases are scattered and replicated in other functions. Moreover, the whole function and all its arguments are executed in a global context. This is less precise and can express fewer properties on individual data objects. For example, we cannot express that the first argument of a function is closed, while its second argument may not be.

Despeyroux et al. [5] presented a type-theoretic foundation for programming with HOAS that supports primitive recursion. To separate data from computation, they introduced modal types $\square A$ that can be injected into computation. However, data is always closed and can only be analyzed by a primitive recursive iterator. Despeyroux and Leleu [4, 3] extended this work to dependent types.

In recent years, various forms of dependent types have found their way into mainstream functional programming languages to allow programmers to express stronger properties about their programs. Generalized algebraic datatypes (GADTs) [26, 2, 29, 16] can index types by other types and have entered mainstream languages such as Haskell. The Dependent ML approach [30] uses indexed types with a fixed set of constraint domains, such as integers with linear inequalities, for which efficient decision procedures exist. However, all these systems lack the power to manipulate and analyze higher-order data. Moreover, they do not support user-defined index domains.

Finally, there have been several proposals in the functional programming community to allow full dependent types in languages such as Cayenne [1] and Epigram [11]. Neither of

these supports HOAS encodings. Moreover, their approach to allowing a user-defined index domain for dependent types is quite different: types can be indexed with arbitrary computations. This is problematic since equality between two types is not simple syntactic equality. Instead, one must first evaluate index arguments. To ensure termination and hence decidability of type checking, Cayenne imposes a heuristic, while Epigram only allows index objects defined by well-founded recursion. This approach violates the idea that type checking should be independent of the operational semantics, and may be costly.

6. CONCLUSION

We have presented an intermediate language for programming with dependently-typed higher-order data. This paper extends the first author's simply-typed work [21] to the dependently-typed setting and lays the foundation for programming with proofs. Our framework distinguishes between implicit data objects that occur in a dependent type and explicit data objects that are recursively analyzed with higher-order pattern matching. We have omitted first-class substitutions from our presentation, a concept present in the simply-typed framework [21], in order to concentrate on the issues related to dependent types, but there is no inherent difficulty in adding them.

We are in the process of completing an implementation of a type checker and interpreter for the internal language described in this paper. However, to make Beluga practical, we plan to address two important questions in the near future. First, we need to reconstruct implicit data objects occurring in computations and data. This follows similar ideas as employed in Twelf [19]. Second, we aim to reconstruct typing annotations at the branches of case expressions. In the long term, we plan to consider the issue of termination checking to ensure that the implemented functions actually do correspond to inductive proofs.

7. REFERENCES

- [1] L. Augustsson. Cayenne—a language with dependent types. In *3rd International Conference on Functional Programming (ICFP '98)*, pages 239–250. ACM, 1998.
- [2] J. Cheney and R. Hinze. First-class phantom types. Technical Report CUCIS TR2003-1901, Cornell University, 2003.
- [3] J. Despeyroux and P. Leleu. Primitive recursion for higher order abstract syntax with dependent types. In *International Workshop on Intuitionistic Modal Logics and Applications (IMLA)*, 1999.
- [4] J. Despeyroux and P. Leleu. Recursion over objects of functional type. *Mathematical Structures in Computer Science*, 11(4):555–572, 2001.
- [5] J. Despeyroux, F. Pfenning, and C. Schürmann. Primitive recursion for higher-order abstract syntax. In *Proceedings of the Third International Conference on Typed Lambda Calculus and Applications (TLCA '97)*, pages 147–163. Springer, 1997. Extended version available as Technical Report CMU-CS-96-172, Carnegie Mellon University.
- [6] J. Dunfield and B. Pientka. Case analysis of higher-order data. Draft available from <http://www.cs.mcgill.ca/~complogic/beluga>, Feb. 2008.
- [7] M. Gabbay and A. Pitts. A new approach to abstract syntax involving binders. In G. Longo, editor, *Proceedings of the 14th Annual Symposium on Logic in Computer Science (LICS'99)*, pages 214–224. IEEE Computer Society Press, 1999.
- [8] A. Gacek, D. Miller, and G. Nadathur. Combining generic judgments with recursive definitions. Accepted to LICS 2008. Draft dated January 21, 2008.
- [9] R. Harper, F. Honsell, and G. Plotkin. A framework for defining logics. *Journal of the ACM*, 40(1):143–184, January 1993.
- [10] D. R. Licata, N. Zeilberger, and R. Harper. Focusing on binding and computation. Technical Report CMU-CS-08-101, Dept. of Computer Science, Carnegie Mellon University, 2008.
- [11] C. McBride and J. McKinna. The view from the left. *Journal of Functional Programming*, 14(1):69–111, 2004.
- [12] A. McCreight and C. Schürmann. A meta-linear logical framework. In *4th International Workshop on Logical Frameworks and Meta-Languages (LFM'04)*, 2004.
- [13] D. Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. *Journal of Logic and Computation*, 1(4):497–536, 1991.
- [14] D. Miller. Unification of simply typed lambda-terms as logic programming. In *Eighth International Logic Programming Conference*, pages 255–269, Paris, France, June 1991. MIT Press.
- [15] A. Nanevski, F. Pfenning, and B. Pientka. Contextual modal type theory. *ACM Transactions on Computational Logic*, 2008. Accepted, to appear.
- [16] S. Peyton Jones, D. Vytiniotis, S. Weirich, and G. Washburn. Simple unification-based type inference for GADTs. In *11th ACM SIGPLAN Int'l Conference on Functional Programming (ICFP '06)*, Sept. 2006.
- [17] F. Pfenning. Unification and anti-unification in the Calculus of Constructions. In *Sixth Annual IEEE Symposium on Logic in Computer Science*, pages 74–85, Amsterdam, The Netherlands, July 1991.
- [18] F. Pfenning. Logical frameworks. In A. Robinson and A. Voronkov, editors, *Handbook of Automated Reasoning*, pages 1063–1147. Elsevier, 2001.
- [19] F. Pfenning and C. Schürmann. System description: Twelf — a meta-logical framework for deductive systems. In H. Ganzinger, editor, *Proceedings of the 16th International Conference on Automated Deduction (CADE-16)*, pages 202–206. Springer-Verlag LNAI 1632, 1999.
- [20] B. Pientka. *Tabled higher-order logic programming*. PhD thesis, Department of Computer Science, Carnegie Mellon University, 2003. CMU-CS-03-185.
- [21] B. Pientka. A type-theoretic foundation for programming with higher-order abstract syntax and first-class substitutions. In *35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'08)*, pages 371–382. ACM, 2008.
- [22] B. Pientka and F. Pfenning. Optimizing higher-order pattern unification. In F. Baader, editor, *19th International Conference on Automated Deduction*,

- Miami, USA, Lecture Notes in Artificial Intelligence (LNAI) 2741, pages 473–487. Springer-Verlag, 2003.
- [23] A. Poswolsky and C. Schürmann. Practical programming with higher-order encodings and dependent types. In *Proceedings of the 17th European Symposium on Programming (ESOP '08)*, Mar. 2008.
- [24] C. Schürmann. *Automating the Meta Theory of Deductive Systems*. PhD thesis, Department of Computer Science, Carnegie Mellon University, 2000. CMU-CS-00-146.
- [25] C. Schürmann, A. Poswolsky, and J. Sarnat. The ∇ -calculus. Functional programming with higher-order encodings. In P. Urzyczyn, editor, *Proceedings of the 7th International Conference on Typed Lambda Calculi and Applications (TLCA'05)*, Nara, Japan, volume 3461 of *Lecture Notes in Computer Science*, pages 339–353. Springer, 2005.
- [26] T. Sheard and E. Pasalic. Meta-programming with built-in type equality. In *Int'l Workshop on Logical Frameworks and Meta-languages (LFM '04)*, pages 106–124, 2004.
- [27] M. R. Shinwell, A. M. Pitts, and M. J. Gabbay. FreshML: programming with binders made simple. In *8th International Conference on Functional Programming (ICFP'03)*, pages 263–274, New York, NY, USA, 2003. ACM Press.
- [28] K. Watkins, I. Cervesato, F. Pfenning, and D. Walker. A concurrent logical framework I: Judgments and properties. Technical Report CMU-CS-02-101, Department of Computer Science, Carnegie Mellon University, 2002.
- [29] H. Xi, C. Chen, and G. Chen. Guarded recursive datatype constructors. In *30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '03)*, pages 224–235, New York, NY, USA, 2003. ACM Press.
- [30] H. Xi and F. Pfenning. Dependent types in practical programming. In *26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'99)*, pages 214–227. ACM Press, 1999.

APPENDIX

Ordinary substitution

In the definition for ordinary data-level substitutions, we need to be careful because the only meaningful data-level terms are those in canonical form. To ensure that substitution preserves canonical form, we use a technique pioneered by Watkins et al. [28] and described in detail in [15]. The idea is to define *hereditary substitution* as a primitive recursive functional that always returns a canonical object.

In the formal development, it is simpler if we can stick to non-dependent types. We therefore first define type approximations α and an erasure operation $()^-$ that removes dependencies. Before applying any hereditary substitution $[M/x]_A^\alpha(B)$ we first erase dependencies to obtain $\alpha = A^-$ and then carry out the hereditary substitution formally as $[M/x]_A^\alpha(B)$. A similar convention applies to the other forms of hereditary substitutions.

Type approximations $\alpha, \beta ::= a \mid \alpha \rightarrow \beta \mid \alpha \times \beta$

Types relate to type approximations via an erasure opera-

tion $()^-$ which we overload to work on types.

$$\begin{aligned} (a \ N_1 \dots N_n)^- &= a \\ (\Pi x:A \dots B)^- &= A^- \rightarrow B^- \\ (\Sigma x:A \dots B)^- &= A^- \times B^- \end{aligned}$$

We can define $[M/x]_\alpha^n(N)$, $[M/x]_\alpha^r(R)$, and $[M/x]_\alpha^s(\sigma)$ by nested induction, first on the structure of the type approximation α and second on the structure of the objects N , R and σ . In other words, we either go to a smaller type approximation (in which case the objects can become larger), or the type approximation remains the same and the objects become smaller. We define the hereditary substitution operations as follows:

$$\begin{aligned} [M/x]_\alpha^n(N) &= N' && \text{Normal terms } N \\ [M/x]_\alpha^r(R) &= R' \text{ or } M' : \alpha' && \text{Neutral terms } R \\ [M/x]_\alpha^s(\sigma) &= \sigma' && \text{Substitutions } \sigma \end{aligned}$$

Data-level normal terms

$$\begin{aligned} [M/x]_\alpha^n(\lambda y. N) &= \lambda y. N' && \text{where } N' = [M/x]_\alpha^n(N) \\ &&& \text{choosing } y \notin \text{FV}(M), \text{ and } y \neq x \\ [M/x]_\alpha^n(M_1, M_2) &= (N_1, N_2) && \text{if } [M/x]_\alpha^n(M_1) = N_1 \\ &&& \text{and } [M/x]_\alpha^n(M_2) = N_2 \\ [M/x]_\alpha^n(R) &= M' && \text{if } [M/x]_\alpha^r(R) = M' : \alpha' \\ [M/x]_\alpha^n(R) &= R' && \text{if } [M/x]_\alpha^r(R) = R' \\ [M/x]_\alpha^n(N) &= \text{fails} && \text{otherwise} \end{aligned}$$

Data-level neutral terms

$$\begin{aligned} [M/x]_\alpha^r(x) &= M : \alpha \\ [M/x]_\alpha^r(y) &= y && \text{if } y \neq x \\ [M/x]_\alpha^r(u[\sigma]) &= u[\sigma'] && \text{where } \sigma' = [M/x]_\alpha^s(\sigma) \\ [M/x]_\alpha^r(p[\sigma]) &= p[\sigma'] && \text{where } \sigma' = [M/x]_\alpha^s(\sigma) \\ [M/x]_\alpha^r(R \ N) &= R' \ N' && \text{where } R' = [M/x]_\alpha^r(R) \\ &&& \text{and } N' = [M/x]_\alpha^n(N) \\ [M/x]_\alpha^r(R \ N) &= M'' : \beta && \text{if } [M/x]_\alpha^r(R) = \lambda y. M' : \alpha_1 \rightarrow \beta \text{ where} \\ &&& \alpha_1 \rightarrow \beta \leq \alpha \text{ and } N' = [M/x]_\alpha^n(N) \\ &&& \text{and } M'' = [N'/y]_{\alpha_1}^n(M') \\ [M/x]_\alpha^r(\text{proj}_i \ R) &= N_i : \alpha_i && \text{if } [M/x]_\alpha^r(R) = (N_1, N_2) : \alpha_1 \times \alpha_2 \\ [M/x]_\alpha^r(\text{proj}_i \ R) &= \text{proj}_i \ R' && \text{if } [M/x]_\alpha^r(R) = R' \\ [M/x]_\alpha^r(R) &= \text{fails} && \text{otherwise} \end{aligned}$$

Data-level substitutions

$$\begin{aligned} [M/x]_\alpha^s(\cdot) &= \cdot \\ [M/x]_\alpha^s(\sigma ; N) &= (\sigma' ; N') && \text{where } \sigma' = [M/x]_\alpha^s(\sigma) \\ &&& \text{and } N' = [M/x]_\alpha^n(N) \\ [M/x]_\alpha^s(\sigma, R) &= (\sigma', R') && \text{if } [M/x]_\alpha^r(R) = R' \\ &&& \text{and } \sigma' = [M/x]_\alpha^s(\sigma) \\ [M/x]_\alpha^s(\sigma, R) &= (\sigma' ; M') && \text{if } [M/x]_\alpha^r(R) = M' : \alpha' \\ &&& \text{and } \sigma' = [M/x]_\alpha^s(\sigma) \\ [M/x]_\alpha^s(\text{id}_\psi) &= \text{id}_\psi \\ [M/x]_\alpha^s(\sigma) &= \text{fails} && \text{otherwise} \end{aligned}$$

We write $\alpha \leq \beta$ and $\alpha < \beta$ if α occurs in β (as a proper subexpression in the latter case). If the original term is not well-typed, a hereditary substitution, though terminating, cannot always return a meaningful term. We formalize this as failure to return a result. However, on well-typed terms, hereditary substitution will always return well-typed terms. This substitution operation can be extended to types for which we write $[M/x]_\alpha^a(A)$.

THEOREM 5 (TERMINATION).

$[M/x]_{\alpha}^*(_)$ where $* = \{n, r, s, a\}$ terminates, either by returning a result or failing after a finite number of steps.

THEOREM 6 (HEREDITARY SUBSTITUTION PRINCIPLES).

If $\Delta; \Psi \vdash M \Leftarrow A$ and $\Delta; \Psi, x:A, \Psi' \vdash J$ then $\Delta; \Psi, [M/x]_{\alpha}^* \Psi' \vdash [M/x]_{\alpha}^*(J)$ where $* = \{n, r, s, a\}$.

Building on the ideas in [15], we can also define simultaneous substitution $[\sigma]_{\bar{\psi}}^n(M)$ (respectively $[\sigma]_{\bar{\psi}}^r(R)$ and $[\sigma]_{\bar{\psi}}^s(\sigma)$). We write $\bar{\psi}$ for the context approximation of Ψ which is defined using the erasure operation $()^-$.

$$\begin{aligned} (\cdot)^- &= \cdot \\ (\psi)^- &= \psi \\ (\Psi, x:A)^- &= (\Psi)^-, x:(A)^- \end{aligned}$$

Substitution for contextual variables

Substitutions for contextual variables are a little more difficult. We have two kinds of contextual variables: meta-variables u and parameter variables p [21].

Contextual substitution for meta-variables.

We define the contextual substitution operations for normal object, neutral objects and substitutions as follows.

$$\begin{aligned} [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(N) &= N' && \text{Normal terms } N \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(R) &= R' \text{ or } M' : \alpha' && \text{Neutral terms } R \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^s(\sigma) &= \sigma' && \text{Substitutions } \sigma \end{aligned}$$

As mentioned earlier, $u[\sigma]$ represents a closure where, as soon as we know which term u should stand for, we can apply σ to it. Because of α -conversion, the variables substituted at different occurrences of u may differ, and we write $\hat{\Psi}.M$ to allow for necessary α -renaming. The contextual substitution is indexed with the type of u . This typing annotation is necessary since we apply the substitution σ hereditarily once we know which term u represents, and hereditary substitution requires the type to ensure termination.

Data-level normal terms

$$\begin{aligned} [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(\lambda y. N) &= \lambda y. N' \text{ where } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n N = N' \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(N_1, N_2) &= (N'_1, N'_2) \\ \text{where } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(N_1) &= N'_1 \text{ and } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(N_2) = N'_2 \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(R) &= R' \text{ where } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(R) = R' \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(R) &= M' \text{ where } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(R) = M' : \beta \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(N) &\text{ fails } && \text{otherwise} \end{aligned}$$

In the following, note that applying $[\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r$ to the closure $u[\sigma]$ first obtains the simultaneous substitution $\sigma' = [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^s \sigma$, but instead of returning $M[\sigma']$, it eagerly applies σ' to M . However before that we recover its domain by $[\sigma'/\bar{\psi}]$. To enforce that we always return a normal object as a result of contextual substitution, we resort to ordinary hereditary substitution. For a thorough explanation, see [15].

Data-level neutral terms

$$\begin{aligned} [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(x) &= x \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(u[\sigma]) &= N : \alpha \text{ where } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \\ &\text{and } [\sigma'/\bar{\psi}]_{\bar{\psi}}^n M = N \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(u'[\sigma]) &= u'[\sigma'] \text{ where } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \\ &\text{choosing } u' \neq u \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(p[\sigma]) &= p[\sigma'] \text{ where } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(R N) &= (R' N') \text{ where } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r R = R' \\ &\text{and } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(N) = N' \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(R N) &= M' : \alpha_2 \\ &\text{if } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r R = \lambda x. M_0 : \alpha_1 \rightarrow \alpha_2 \text{ for } \alpha_1 \rightarrow \alpha_2 \leq \alpha[\bar{\psi}] \\ &\text{and } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^n(N) = N' \text{ and } [N'/x]_{\alpha_1}^n(M_0) = M' \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(\text{proj}_i R) &= \text{proj}_i R' \text{ if } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(R) = R' \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(\text{proj}_i R) &= M_i : \alpha_i \\ &\text{if } [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(R) = (M_1, M_2) : \alpha_1 \times \alpha_2 \\ [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^r(R) &\text{ fails } && \text{otherwise} \end{aligned}$$

Contextual substitution for parameter variables.

Contextual substitution for parameter variables follows similar principles, but substitutes an ordinary variable for a parameter variable. We write parameter substitutions as $[\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^*$ where $* \in \{n, r, s, a\}$. When we encounter a parameter variable $p[\sigma]$, we replace p with the ordinary variable x and apply the substitution $[\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^s$ to σ obtaining a substitution σ' . Instead of returning a closure $x[\sigma']$ as the final result we apply σ' to the ordinary variable x . This may again yield a normal term, so we must ensure that contextual substitution for parameter variables preserves normal forms.

$$\begin{aligned} [\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^r(p[\sigma]) &= M : \alpha \text{ if } [\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \\ &\text{and } [\sigma'/\bar{\Psi}]_{\bar{\psi}}^r x = M : \alpha \\ [\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^r(p[\sigma]) &= R \text{ if } [\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \\ &\text{and } [\sigma'/\bar{\Psi}]_{\bar{\psi}}^r x = R \\ [\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^r(p'[\sigma]) &= p'[\sigma'] \text{ where } [\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \end{aligned}$$

THEOREM 7 (TERMINATION).

$[\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^*(_)$ and $[\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^*(_)$ where $* \in \{n, r, s, a\}$ terminate, either by returning a result or failing after a finite number of steps.

THEOREM 8 (CONTEXTUAL SUBSTITUTION PRINCIPLES).

- If $\Delta_1; \Phi \vdash M \Leftarrow A$ and $\Delta_1, u::A[\Phi], \Delta_2; \Psi \vdash J$ then $\Delta_1, [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^* \Delta_2; [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^* \Psi \vdash [\hat{\Psi}.M/u]_{\alpha[\bar{\psi}]}^* J$ where $* = \{n, r, s, a\}$.
- If $\Delta_1; \Phi \vdash x \Rightarrow A$ and $\Delta_1, p::A[\Phi], \Delta_2; \Psi \vdash J$ then $\Delta_1, [\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^* \Delta_2; [\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^* \Psi \vdash [\hat{\Psi}.x/p]_{\alpha[\bar{\psi}]}^* J$ where $* = \{n, r, s, a\}$.